A

Special Project Mid-term Report

on

AI Voice Controlled ERP Assistant

by

Pranamya Mani Tripathi (24STUCHH010777)
Gone Srishanth Reddy (24STUCHH010428)
Harshulchandhra (24STUCHH010172)
Chintakindi Badrinath Goud (24STUCHH010308)

Under the guidance of
Sowjanya Ramisetty Mam



Faculty of Science & Technology,
The ICFAI Foundation for Higher Education, Hyderabad
September, 2026

Special Project: Literature Review, Abstract and Project Outline

AI Voice-Controlled ERP Assistant: A Natural Language Interface for ERP Operations

Literature Review

1. Introduction

Topic area. Enterprise Resource Planning (ERP) systems integrate inventory, human resources, attendance and sales data in one platform, yet they are typically operated through many screens, menus and forms. This project sits at the intersection of conversational AI, speech technology and enterprise software. It builds a natural language interface in which a user speaks or types a command such as *"Show stock for item P-102"* and the system understands it, asks for missing details, confirms any change to data, and returns the result as text or speech.

Problem statement. Routine ERP tasks require multi-step navigation, form filling and knowledge of menu hierarchies. This is slow and error-prone, and it creates a steep learning curve for non-technical users. The project asks: *how can users perform common ERP operations through voice or text in plain language, while keeping data writes safe and intentional?*

Selection criteria. Articles were selected on three criteria: (i) *relevance* to a component of the proposed pipeline (speech recognition, intent detection and entity extraction, dialogue management, or conversational frameworks); (ii) *credibility*, meaning peer-reviewed journals or major conference proceedings or widely used open-source technical reports; and (iii) *publication year*, covering foundational work (2013–2017) and recent work (2019–2023) so that both established principles and current tools are represented.

2. Previous Works and Studies

Article 1

Title	Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)
Authors	A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, I. Sutskever
Source	Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202, pp. 28492–28518
Year	2023
Methodology / Tools	Encoder–decoder Transformer trained on 680,000 hours of multilingual, multitask audio–transcript data collected from the internet. Evaluated zero-shot, with no dataset-specific fine-tuning.

Key Findings	The models generalise well to standard benchmarks, are competitive with fully supervised systems and approach human accuracy and robustness. Released as open models, they are practical for real applications such as voice assistants.
Limitations	Larger models need considerable compute, so latency and memory can be a problem on modest hardware. Accuracy on domain-specific terms such as item codes (for example “P-102”) is not guaranteed.

Article 2

Title	Attention-Based Recurrent Neural Network Models for Joint Intent Detection and Slot Filling
Authors	B. Liu, I. Lane
Source	Proceedings of Interspeech 2016 (ISCA), pp. 685–689
Year	2016
Methodology / Tools	Attention-based encoder–decoder and alignment-based RNN models trained jointly for intent classification and slot (entity) labelling. Benchmarked on the ATIS air-travel dataset.
Key Findings	Joint training reduced intent detection error by 0.56% absolute (23.8% relative) and improved slot filling F1 over independent models, showing that intent and entities should be modelled together.
Limitations	Needs large labelled datasets per domain. The evaluation uses a single, narrow benchmark (ATIS) and does not cover dialogue, confirmation or execution against a backend.

Article 3

Title	BERT for Joint Intent Classification and Slot Filling
Authors	Q. Chen, Z. Zhuo, W. Wang
Source	arXiv preprint arXiv:1902.10909
Year	2019
Methodology / Tools	Fine-tunes a pre-trained BERT model with a joint classification head for intent and a token-level head for slots, tested on standard spoken language understanding benchmarks.
Key Findings	Pre-trained language models significantly improve intent and slot accuracy and cope better with varied phrasing than earlier models, and joint fine-tuning beats separate models.
Limitations	Requires fine-tuning data and GPU resources. It is an isolated NLU component, with no handling of missing slots, multi-turn context or safe database writes.

Article 4

Title	Rasa: Open Source Language Understanding and Dialogue Management
Authors	T. Bocklisch, J. Faulkner, N. Pawlowski, A. Nichol
Source	arXiv preprint arXiv:1712.05181 (technical report)
Year	2017
Methodology / Tools	Modular open-source framework: NLU pipeline for intent and entity extraction plus a machine-learning dialogue policy (Rasa Core) that predicts the next action from conversation history.
Key Findings	Shows that a practical assistant is best built as separate NLU and dialogue components that developers can configure, extend and connect to custom actions and backends.
Limitations	General-purpose and fairly heavy to configure. It has no built-in ERP-specific write safeguards, and its training-based approach needs example conversations.

Article 5

Title	POMDP-Based Statistical Spoken Dialog Systems: A Review
Authors	S. Young, M. Gašić, B. Thomson, J. D. Williams
Source	Proceedings of the IEEE, vol. 101, no. 5, pp. 1160–1179
Year	2013
Methodology / Tools	Survey of statistical dialogue management, formulating dialogue as a partially observable Markov decision process that tracks belief over user goals despite speech-recognition errors.
Key Findings	Explains why dialogue managers must be robust to noisy input: tracking uncertainty and choosing to clarify or confirm reduces errors compared with acting on the top recognition hypothesis.
Limitations	Statistical policies require training data or simulated users and are complex to build. Simpler deterministic policies are more practical for small, well-defined tasks.

3. Research Gaps / Limitations Identified

Limitations in previous systems and approaches:

- Speech recognition and NLU research (Articles 1–3) is usually evaluated on benchmarks in isolation. It seldom shows a complete pipeline from spoken command to an actual enterprise database operation.
- Neural NLU models depend on labelled data, fine-tuning and GPUs, which is a barrier for students, small organisations and laptop-only environments.
- General frameworks (Article 4) are flexible but heavy, and they do not provide ready-made safeguards such as mandatory confirmation before an ERP write, or role-based access checks.

- Statistical dialogue management (Article 5) addresses uncertainty in principle, but is complex for a focused task with a small set of ERP intents.
- Users normally have to know exact record codes, while ERP data is referred to in everyday speech by names (for example “hydraulic valve”).

Opportunities this project addresses:

- A complete, working pipeline (input, STT, NLU, dialogue manager, ERP gateway, database, response, TTS) that runs on an ordinary laptop.
- A dual NLU design: a fast, deterministic rule-based engine by default, and an optional local LLM engine (Ollama with Llama 3.2) for natural variation, with automatic fallback to rules.
- A dialogue manager that asks for missing entities, asks the user to confirm uncertain entity matches, and requires explicit confirmation before any write operation.
- Fuzzy entity resolution so that users can refer to items and employees by name, and pluggable STT and TTS engines that can be swapped without changing the core system.

4. Summary / Conclusion of Review

The literature shows that each building block of a voice-controlled ERP assistant is mature: open speech recognition such as Whisper is robust enough for practical use; joint intent and slot models, from attention-based RNNs to BERT, are the standard way to interpret commands; and modular frameworks and dialogue-management research show that clarification and confirmation are essential when inputs are noisy. What is missing is a lightweight, integrated and safety-aware system that connects these pieces to real ERP operations. This review therefore supports the design of the present project: a layered architecture (interface, AI processing, integration, ERP core) that separates NLU from dialogue management, keeps components swappable, uses a rule-based NLU with an optional LLM upgrade to remain runnable on limited hardware, and treats write confirmation as a core requirement.

Abstract

Introduction. Enterprise Resource Planning (ERP) systems are the operational backbone of many organisations, holding inventory, employee, attendance and sales records. Despite their power, they are difficult to use: even a routine query requires navigating several screens, completing forms and understanding complex menu structures. Recent advances in speech recognition and natural language understanding make it possible to interact with such systems through ordinary language.

Problem Statement. Traditional ERP navigation is slow, multi-step and error-prone. Repetitive operations consume disproportionate time, and non-technical users face a steep learning curve. Existing speech and NLU research is mostly evaluated in isolation and rarely delivers a complete, safe path from a spoken command to an ERP database operation.

Purpose. The purpose of this project is to design and implement an AI Voice-Controlled ERP Assistant that lets users perform ERP functions using natural language, by voice or text, instead of manual navigation. The system aims to support inventory queries, attendance operations, sales reports and employee information, while ensuring that data is changed only with the user's explicit confirmation.

Methodology. The system follows a four-layer architecture: a web-based user interface (HTML, CSS, JavaScript) accepting voice and text; an AI processing layer for speech-to-text (Whisper or Vosk), intent and entity extraction, and dialogue management; an integration layer built on a FastAPI backend with an ERP gateway exposing REST endpoints; and an ERP core implemented as a mock SQLite database with Inventory, Employees, Attendance and Sales modules. Requests pass through a seven-stage pipeline: input, speech-to-text, NLU, dialogue manager, ERP gateway, database query and response (text, with optional speech output through pyttsx3). NLU is rule-based (regex pattern matching) by default, with an optional LLM-based mode using Ollama and Llama 3.2. Fuzzy matching resolves names such as "hydraulic valve" to item codes, and the dialogue manager handles missing-information clarification and write confirmation.

Expected Output / Results. The expected output is a working prototype with a browser interface and a FastAPI backend that correctly handles the core use cases: "Show stock for item P-102", "Mark my attendance" (with confirmation), "Get total sales from last month" and multi-turn clarification such as "Check stock" followed by "Which item?" and "P-101". The project will also deliver documentation and a test evaluation of intent accuracy, entity resolution and response time for both NLU modes. The design is intended to allow the mock SQLite database to be replaced by a production ERP such as Odoo or a PostgreSQL-backed system, with authentication, multilingual support and cloud deployment as future work.

Keywords: ERP, voice assistant, natural language understanding, intent and entity extraction, dialogue management, speech recognition, FastAPI.

Project Outline

1. Project Title

AI Voice-Controlled ERP Assistant: A Natural Language Interface for ERP Operations.

2. Objective(s)

The overall goal is to enable users to interact with ERP functions using natural language (voice or text) instead of manually navigating the system. Specific objectives:

- Design a layered architecture (UI, AI processing, integration, ERP core) with a clear responsibility for each layer.
- Implement intent recognition and entity extraction using a rule-based engine, with an optional LLM-based engine for natural variation.
- Build a dialogue manager that asks for missing information, confirms uncertain entities and requires confirmation before every write.
- Provide fuzzy entity resolution so that users can refer to records by name rather than exact code.
- Support voice input (Whisper or Vosk) and speech output (pyttsx3) through pluggable engines, alongside text input.
- Evaluate the prototype for correctness, usability and response time.

3. Scope

Included:

- Web interface with text input and a voice record button, and a FastAPI backend with REST endpoints (text and audio).
- Mock SQLite ERP with four modules: Inventory, Employees, Attendance and Sales.
- Use cases: stock query, attendance marking and retrieval (with confirmation), sales report by date range, employee information, and the clarification flow.
- Rule-based NLU (default) and optional Ollama + Llama 3.2 NLU with fallback; fuzzy entity resolution; role-based access hook in the gateway; optional login using hashed credentials.

Excluded (future work):

- Connection to a production ERP (Odoo, SAP or PostgreSQL), full OAuth2/JWT authentication, rate limiting and security hardening.
- Multilingual support, proactive alerts (low stock, attendance anomalies) and Docker/cloud deployment.

Key deliverables: working source code, demo script with interactive mode, web interface, project report and presentation slides.

4. Project Timeline

The plan below is arranged in modules over ten weeks. Adjust the week numbers so that final submission meets the official project submission deadline.

Weeks	Module / Phase	Tasks and Milestone
1	Planning and review	Finalise problem statement, complete literature review, define use cases and architecture. Milestone: approved outline.
2	ERP core	Design SQLite schema (inventory, employees, attendance, sales), seed sample data, write database functions.
3–4	NLU module	Define intents and required entities; build regex-based intent classifier and entity extractor; unit-test on sample commands.
5	Entity resolution and dialogue manager	Implement fuzzy matching; clarification loop, misheard-entity confirmation and write confirmation. Milestone: text pipeline works end to end.
6	Integration layer	Build FastAPI server, ERP gateway, REST endpoints, response formatting and role-based access hook.
7	Speech I/O and web interface	Integrate Whisper/Vosk STT and pyttsx3 TTS; build browser UI with chat and record button.
8	Optional LLM NLU	Integrate Ollama + Llama 3.2 with fallback to rule-based NLU; compare both modes.
9	Testing and evaluation	Test intents, entities, clarification and confirmation flows; measure accuracy and latency; fix defects.
10	Documentation and submission	Write final report, prepare slides and demo, submit before the deadline.

5. Resources and Tools

Software:

- Language and backend: Python, FastAPI, Uvicorn, Pydantic, python-multipart, requests.
- Frontend: HTML, CSS and JavaScript (no build step).
- Database: SQLite (mock ERP schema).
- Speech: Whisper (API or local) or Vosk for speech-to-text; pyttsx3 for text-to-speech.
- Optional NLU: Ollama with Llama 3.2 (1B model, runs on CPU).
- Development: VS Code or similar editor, Git, a modern web browser, Swagger UI (/docs) for API testing.

Hardware:

- A Windows laptop with a working microphone and speakers (a CPU-only machine is sufficient; the 1B LLM and text mode keep requirements low).
- Internet connection only for optional Whisper API use and one-time model downloads.

Data: A synthetic SQLite database with sample items, employees, attendance and sales records, plus a set of test commands (no external dataset required).

6. Methodology / Approach

The contribution is an end-to-end, layered assistant built incrementally, with each component tested before integration:

- **Input:** the user speaks or types a command in the web interface. Text bypasses speech-to-text and enters the pipeline at the NLU stage.
- **Speech-to-text:** audio is converted to text by Whisper or Vosk through a pluggable interface, with audio preprocessing.
- **NLU:** the intent (for example Stock Query) and entities (for example P-102) are extracted, by regex patterns by default or by an Ollama-hosted Llama 3.2 model when enabled, falling back to rules if the LLM is unavailable.
- **Entity resolution:** fuzzy matching maps names to ERP records; uncertain matches trigger a clarification prompt instead of a silent guess.
- **Dialogue manager:** checks completeness and reliability, asks for missing entities, and requires "yes" before any write (such as marking attendance).
- **ERP gateway and database:** the validated request is routed by FastAPI to the right ERP function, which queries or updates SQLite.
- **Response:** the result is formatted as text and optionally spoken through pyttsx3.
- **Evaluation:** test cases per intent, including phrasing variations, missing-entity and confirmation flows; measure intent accuracy, entity resolution accuracy and response time for rule-based and LLM modes.

7. Conclusion

The project aims to show that a layered, safety-aware natural language interface can make ERP operations accessible without manual navigation or training. Success will be measured by: (i) all demonstrated use cases (stock query, attendance with confirmation, sales report, clarification flow) working end to end; (ii) correct intent and entity recognition on a test set of commands, with the LLM mode handling more phrasing variation than the rule-based mode; (iii) no database write without user confirmation; and (iv) acceptable response time on a standard laptop. Further extensions include integration with a real ERP such as Odoo or SAP, OAuth2/JWT authentication, multilingual voice support, proactive alerts, and Docker or cloud deployment.